

Building the Trivia Game

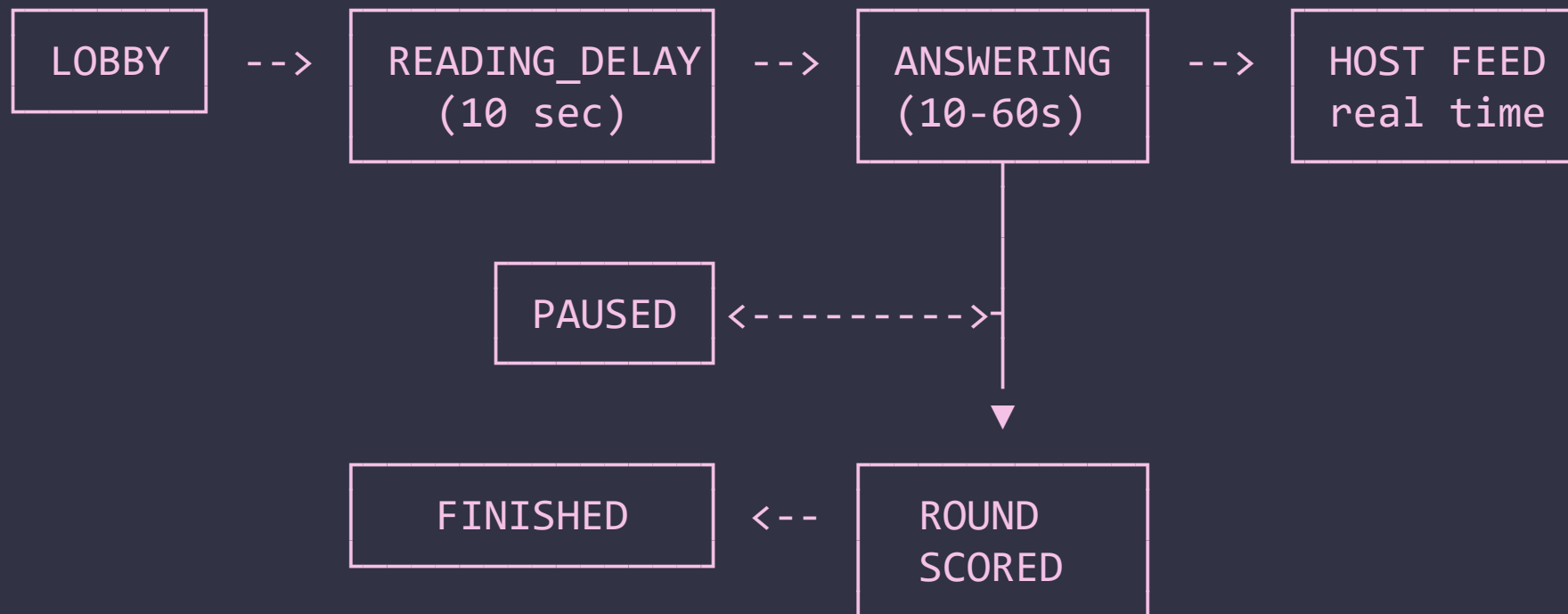
How did I do something I had no business doing in like 8 hours?

A real-time multiplayer trivia platform, architecture & design overview

What are some of the Game Features 🎮

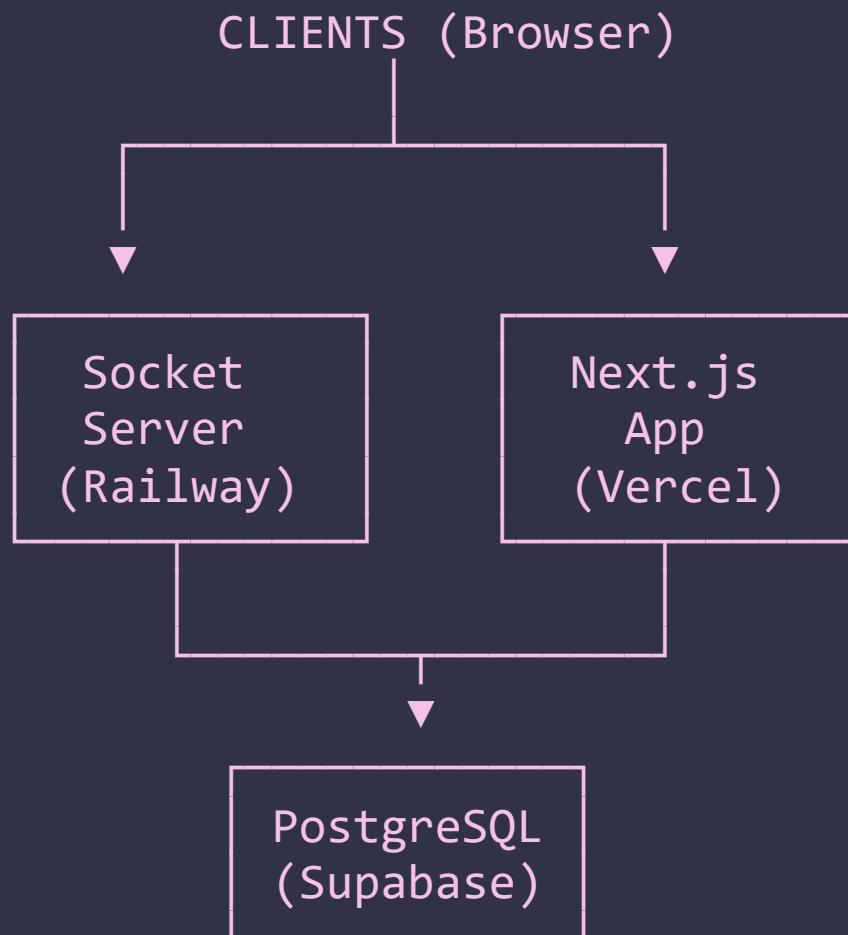
Feature	Implementation
Fully configurable games	Questions & answers loaded with a CSV, timer at launch
Hardest Questions	Auto-calculates lowest correct rate & displays
Live leaderboard	Updates after each round
Separate host/player clients	Different UI/UX for each
Response time tracking	Potential for tiebreakers
Team names	Customizable per team
Completely web-based	No downloads, works on any device
Guest access	No auth/signup needed, easy QR code

Game Flow State Machine





System Architecture



```
// Next.js 14 + react 18 + TypeScript  
// Tailwind CSS for styling  
// Socket.IO Client for real-time comms
```

```
// When new features/bugs are pushed:  
// <--- Fed by Github <--- Fed by VSCode
```

🧑 What are these components: Vercel

- Cloud-based software to deploy and host modern frontend frameworks and static websites with easy config.
- Provided great feedback on compiler issues when setting up.
- Allowed for multiple environment settings (via config parameters).
- Has a lot of monitoring / environment scaffolding.

The screenshot shows the Vercel dashboard for a project named "trivia-game-project-5wwn". The top navigation bar includes a search bar, a "Feedback" button, and a user profile icon. Below the navigation bar, there's a section for "trivia-game-project-5wwn" with buttons for "Repository", "Usage", "Domains", and "Visit". The main content area is titled "Production Deployment" and features a "Build Logs" button, a "Runtime Logs" button, and an "Instant Rollback" button. A preview of the deployed application is shown, displaying a "Trivia Game" interface with a "Join a Game" button and a "Host a Game" button. To the right of the preview, deployment details are listed: "trivia-game-project-5wwn-gkvojwg44-stephen-schwartzs-projects.vercel.app", "Domains" (trivia-game-project-5wwn.vercel.app), "Status" (Ready), and "Source" (main). Below the deployment details, there's a "Deployment Settings" section with "Recommendations" and "Build Settings". The "Recommendations" section includes three cards: "Build Multiple Deployments Simultaneously", "Get builds up to 40% faster", and "Prevent Frontend-Backend Mismatches". The "Build Settings" section includes "On-Demand Concurrent Builds" (Disabled), "Build Machine" (Standard, 4 vCPUs, 8 GB Memory), and "Prioritize Production Builds" (Enabled). The "Runtime Settings" section includes "Fluid Compute" (Enabled), "Function CPU" (Standard, 1 vCPU, 2 GB Memory), "Node.js Version" (24.x), "Deployment Protection" (Standard Protection), "Skew Protection" (Disabled), and "Cold Start Prevention" (Disabled).

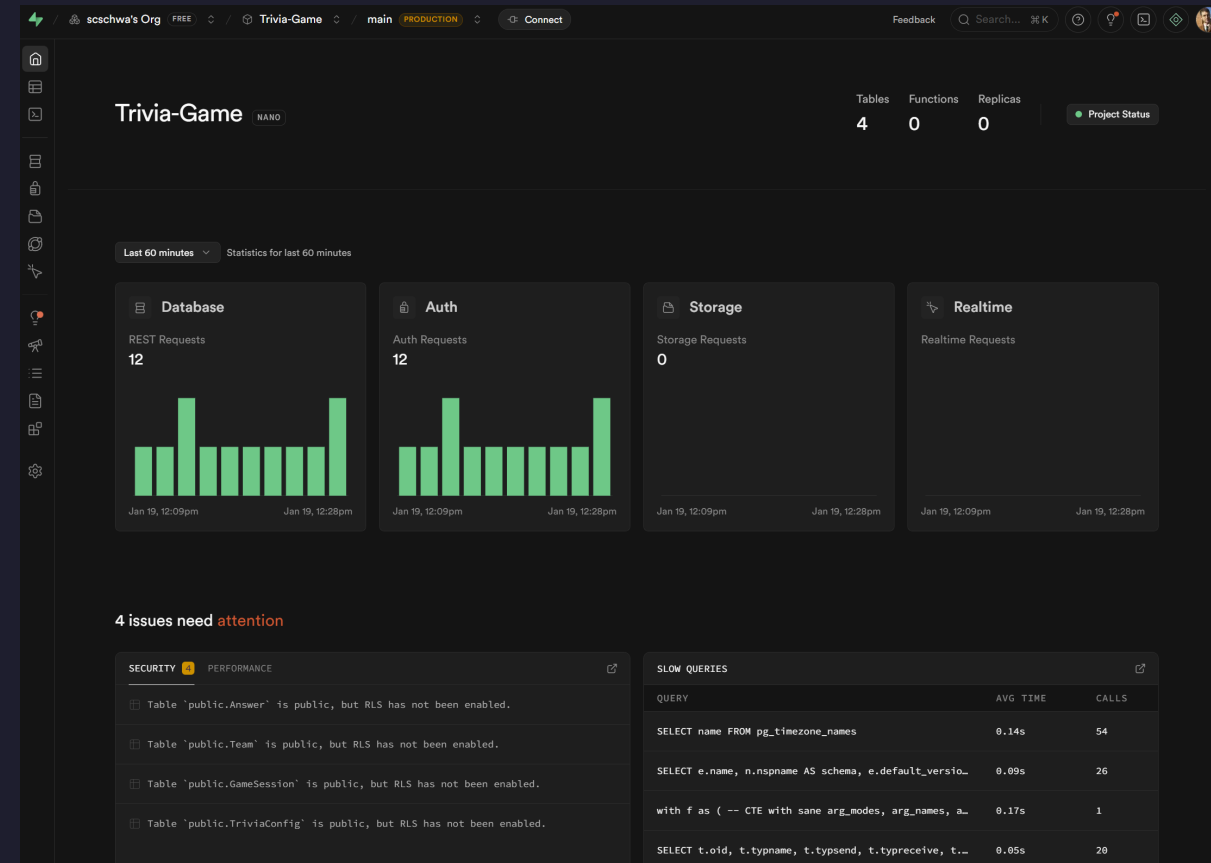
🧑 What are these components: Railway.app

- Only a portion of the project is deployed here (socket-server).
- A back-end equivalent of Vercel. Allows for auto-scaling with each inbound request of the socket server.
- Database connectivity is pooled, with direct multi-client write/read enabled.
- \$5 spend if you have an older github account to link to it.

The screenshot displays the Railway.app dashboard for the 'trivia-game-project'. The top navigation bar includes the project name 'adequate-truth', environment 'production', and various settings like 'Architecture', 'Observability', 'Logs', and 'Settings'. A green badge indicates '24 days or \$4.90 left'. The main panel shows the 'trivia-game-project' deployment details, including the URL 'trivia-game-project-production.up.railway.app', the image 'node@22.22.0', the region 'us-west2', and '1 Replica'. The deployment is marked as 'ACTIVE'. A table lists the deployment steps: 'Initialization' (00:00), 'Build' (00:25), 'Deploy' (00:05), and 'Post-deploy' (00:00), all with green checkmarks indicating success. Below this, a 'HISTORY' section lists several previous deployments, each marked as 'REMOVED' and dated '2 days ago via GitHub'.

🧑 What are these components: Supabase

- Open-source database-as-a-service (DaaS, but the good kind, not like CDA-EU 😊).
- This lets me build, host, and connect a PostgreSQL database with minimal setup.
- It also provides the appropriate connection pooling needed for serverless environments like Vercel/Railway.
- Has a really nice dashboard for running queries, inspecting data, and managing roles.



Why This Architecture?

Decision	Rationale
Separate Socket Server	Vercel doesn't support persistent WebSockets
Server-Authoritative Timers	Ensures sync with dynamic offsets (was having issues w/ this)
PostgreSQL	Required for serverless (Supabase pooling features were clutch)
Next.js Server Actions	Type-safe mutations, showing any potential errors upfront, this got a lot of debugging out of the way.
Prisma ORM	Shared schema, but type-safe DB to avoid problems/conflicts that could crash clients unexpectedly, ruining gameplay



Lines of Code by Language

Language	Lines	Files	Purpose
TypeScript/TSX	7,819	46	Application logic, React components
CSS	4,846	2+	Styling (Tailwind + custom)
Prisma Schema	~290	2	Database models
JSON Config	327	6	Package & TypeScript config
Documentation	709	4+	README, guides
Total	~14,000	~60	

I wrote **basically zero** of it (besides the game questions and this presentation).

Frontend Dependencies (18 packages)

Category	Packages
Framework	<code>next</code> (14.1), <code>react</code> (18.2), <code>react-dom</code>
Database	<code>@prisma/client</code> (5.10)
UI Components	<code>lucide-react</code> , <code>recharts</code> , <code>qrcode.react</code>
Forms	<code>react-hook-form</code> , <code>zod</code>
File Handling	<code>papaparse</code> (CSV), <code>react-dropzone</code>
Real-time	<code>socket.io-client</code>
Auth	<code>bcrypt</code> , <code>jose</code> (JWT)

I couldn't have navigated this without significant tutorials. I had never used `npm install` or `npx prisma *` before starting this project.

Socket Server Dependencies (8 packages)

Category	Packages
Server	<code>express</code> , <code>socket.io</code> , <code>cors</code>
Database	<code>@prisma/client</code>
Auth	<code>bcrypt</code> , <code>jose</code>
Config	<code>dotenv</code>
Validation	<code>zod</code>

Ditto above - I had never built a socket server before, so this was all new to me.

⚡ Technical Highlights for Features I Added

1. Clock Skew Compensation

- `serverTime` offset syncs timers across devices

2. Reconnection Support

- Unique `reconnectToken` in local client storage (cookies)

3. Archive vs Delete

- Unused configs deletable
- Used configs archived (was less impactful to database)

iv. QR Code Joining

- Easy mobile player onboarding!

4. Keyboard Shortcuts

- Host controls: Space, P, N, S, L, Esc

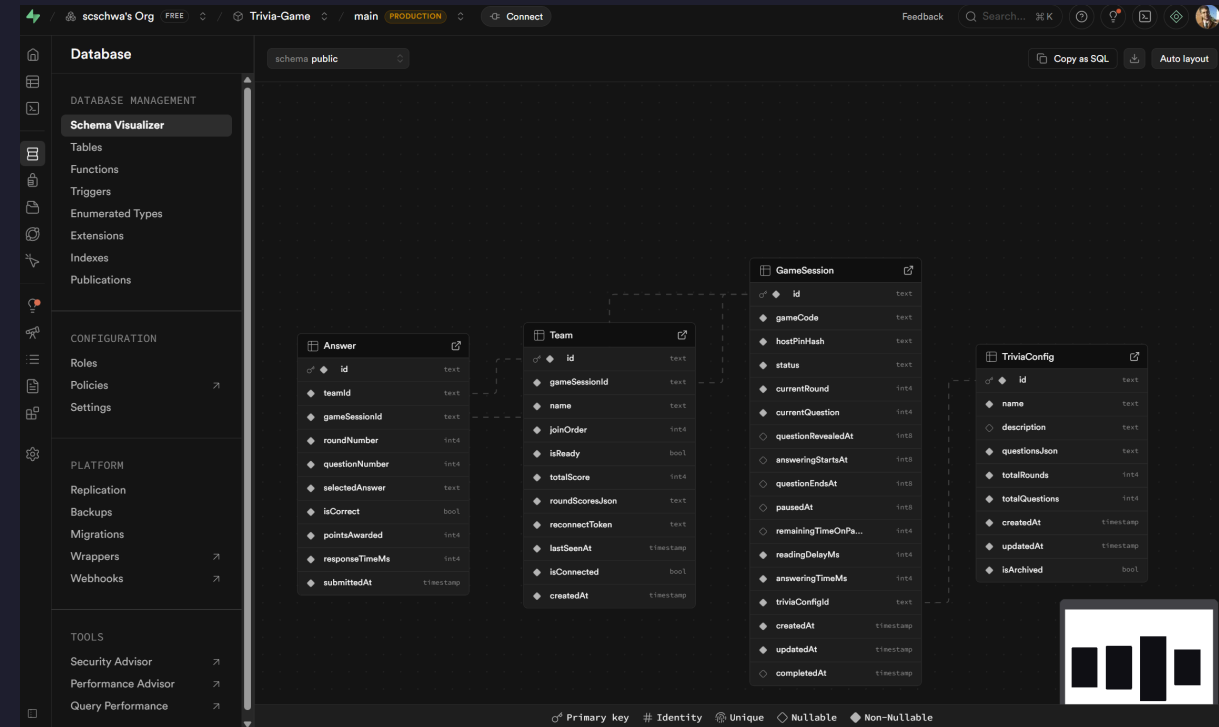


Database Design

- Questions as JSON blob - Faster reads, simpler schema
- Per-round scores - Enables breakdown analytics
- Strategic indexes - gameCode, status, createdAt

4 Table Models:

TriviaConfig → GameSession → Team → Answer



Security Measures

-  Host PIN hashed with **bcrypt** for secure play
-  Unique team **reconnect tokens** for session integrity
-  Rate limiting: **5 events/sec** per socket avoids spamming
-  Input validation with **Zod schemas** which prevents data issues

Development Statistics

Metric	Value
Total Commits	33
Development Period	5 days
Lines Inserted	23,599
Lines Deleted	264
Net Lines Added	~23,335
Average Commits/Day	6.6

Most of this was up and running in about 2-3 hours from the initial prompt. I spent another 4-5 hours debugging, polishing, and adding features, and doing some playthrus on multiple clients.

Commit Breakdown

Category	Count	%
Bug Fixes	18	55%
Features	6	18%
Infrastructure	4	12%
Documentation	3	9%
Other	2	6%

Almost all commits (besides 1st one) were small and focused, with the majority being performed by Claude Opus 4.5.



Traditional Development Estimate

How long should this have taken (according to GPT 5.2)?

Component	Junior	Mid-Level	Senior
Database schema & Prisma setup	2 days	1 day	0.5 days
Next.js frontend (pages, components)	5-7 days	3-4 days	2 days
Socket server (real-time logic)	4-5 days	2-3 days	1-2 days
Timer synchronization system	2-3 days	1-2 days	1 day
Auth & security (PIN, tokens)	2 days	1 day	0.5 days
CSV upload & validation	1-2 days	1 day	0.5 days
Deployment & debugging	3-4 days	2 days	1 day
UI polish & styling	3-4 days	2 days	1-2 days
Total	4-6 weeks	2-3 weeks	1-2 weeks

Development Time Comparison

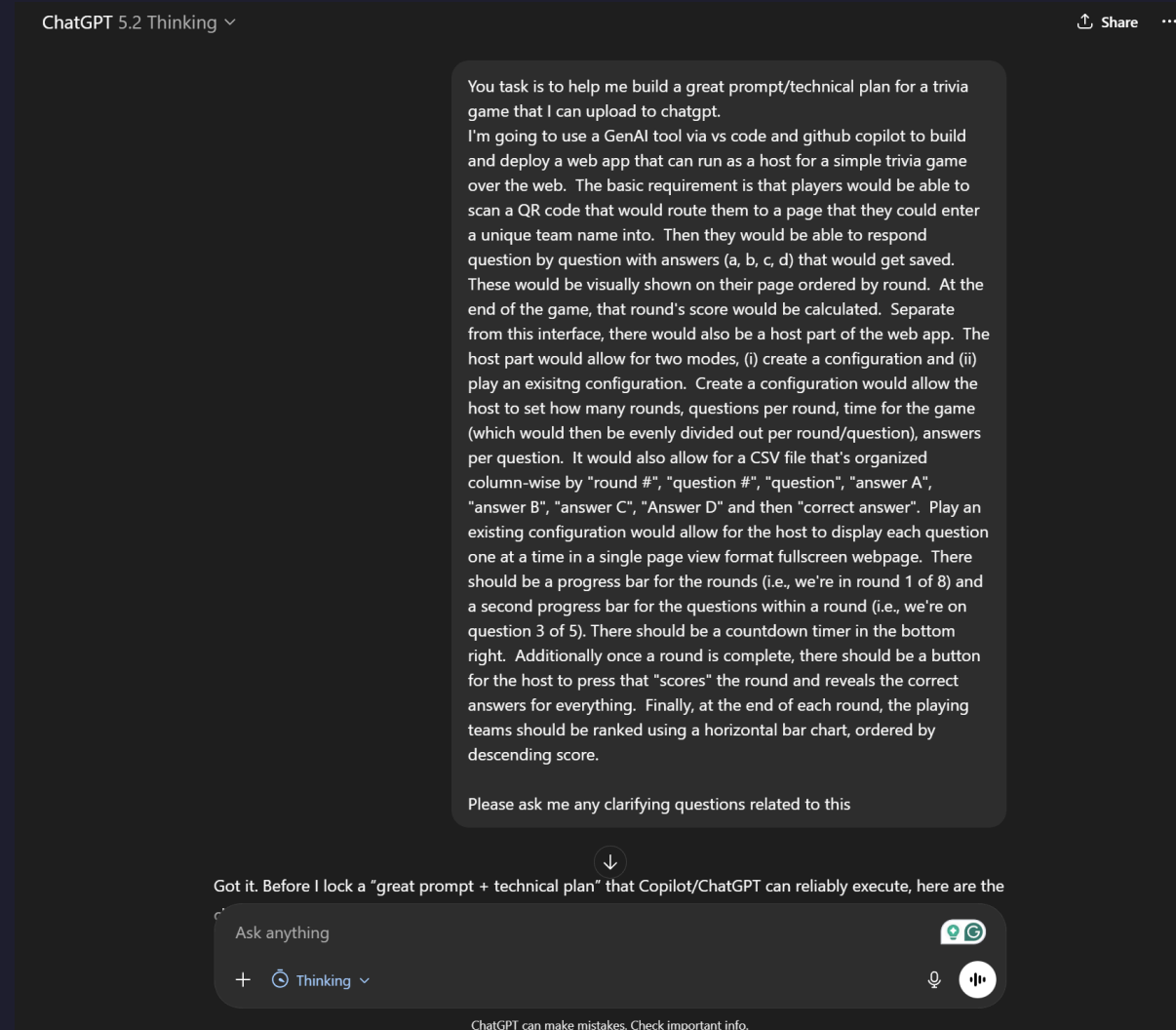
🕒 Development Time: AI-Assisted

Metric	Value
Actual time spent	~8 hours
Calendar span	5 days
Lines of code	~14,000
Characters	1,801,714
Tokens	450,428

~10-20x speedup with GHCP and Claude 4.5 Opus AI vs. a Mid-level / Senior Developer.

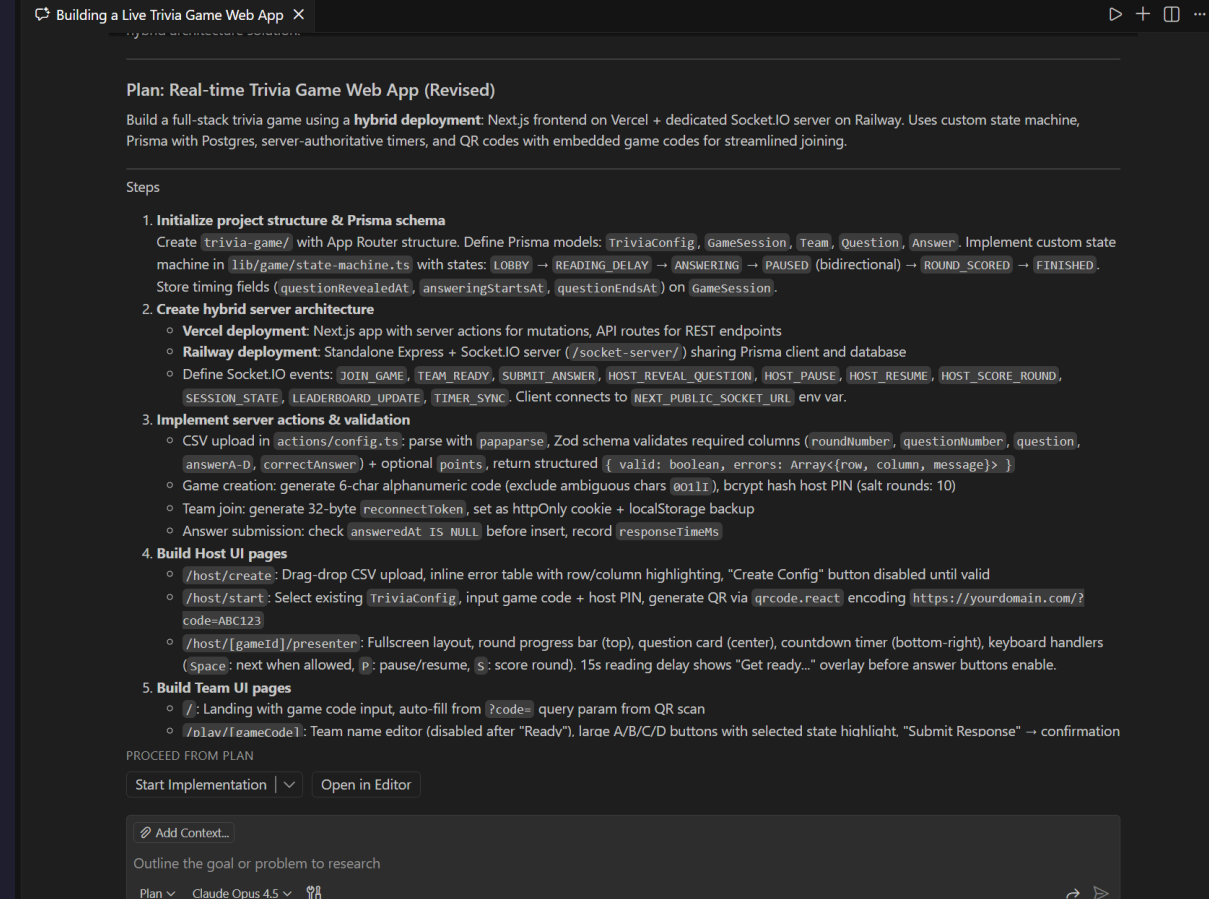
What was my workflow?

- I started with ChatGPT, 5.2 Thinking with this prompt.
- It asked 23 follow-up questions to clarify requirements, which I provided.
- This generated a markdown-based implementation spec, which I could easily pass into GHCP via VSCode.



What was my workflow? (con't)

- I am using VSCode 1.108.0 (December 2025) with GHCP Copilot extension.
- I made a new project folder, launch VSCode, dropped in this imp-spec and asked GHCP in `planning` mode to create an imp-plan.
- Once satisfied and done with questions, I asked it to implement.



The screenshot shows a VS Code window titled "Building a Live Trivia Game Web App". The main content area displays a project plan for a "Real-time Trivia Game Web App (Revised)". The plan describes a full-stack trivia game using a hybrid deployment: Next.js frontend on Vercel and a dedicated Socket.IO server on Railway. It mentions a custom state machine, Prisma with Postgres, server-authoritative timers, and QR codes for streamlined joining.

Plan: Real-time Trivia Game Web App (Revised)

Build a full-stack trivia game using a **hybrid deployment**: Next.js frontend on Vercel + dedicated Socket.IO server on Railway. Uses custom state machine, Prisma with Postgres, server-authoritative timers, and QR codes with embedded game codes for streamlined joining.

Steps

- 1. Initialize project structure & Prisma schema**
 - Create `trivia-game/` with App Router structure. Define Prisma models: `TriviaConfig`, `GameSession`, `Team`, `Question`, `Answer`. Implement custom state machine in `lib/game/state-machine.ts` with states: `LOBBY` → `READING_DELAY` → `ANSWERING` → `PAUSED` (bidirectional) → `ROUND_SCORED` → `FINISHED`. Store timing fields (`questionRevealedAt`, `answeringStartsAt`, `questionEndsAt`) on `GameSession`.
- 2. Create hybrid server architecture**
 - `Vercel deployment`: Next.js app with server actions for mutations, API routes for REST endpoints
 - `Railway deployment`: Standalone Express + Socket.IO server (`/socket-server/`) sharing Prisma client and database
 - Define Socket.IO events: `JOIN_GAME`, `TEAM_READY`, `SUBMIT_ANSWER`, `HOST_REVEAL_QUESTION`, `HOST_PAUSE`, `HOST_RESUME`, `HOST_SCORE_ROUND`, `SESSION_STATE`, `LEADERBOARD_UPDATE`, `TIMER_SYNC`. Client connects to `NEXT_PUBLIC_SOCKET_URL` env var.
- 3. Implement server actions & validation**
 - CSV upload in `actions/config.ts`: parse with `papaparse`, Zod schema validates required columns (`roundNumber`, `questionNumber`, `question`, `answerA-D`, `correctAnswer`) + optional `points`, return structured `{ valid: boolean, errors: Array<{row, column, message}> }`
 - Game creation: generate 6-char alphanumeric code (exclude ambiguous chars `0011I`), bcrypt hash host PIN (salt rounds: 10)
 - Team join: generate 32-byte `reconnectToken`, set as httpOnly cookie + `localStorage` backup
 - Answer submission: check `answeredAt IS NULL` before insert, record `responseTimeMs`
- 4. Build Host UI pages**
 - `/host/create`: Drag-drop CSV upload, inline error table with row/column highlighting, "Create Config" button disabled until valid
 - `/host/start`: Select existing `TriviaConfig`, input game code + host PIN, generate QR via `qrcode.react` encoding `https://yourdomain.com/?code=ABC123`
 - `/host/[gameId]/presenter`: Fullscreen layout, round progress bar (top), question card (center), countdown timer (bottom-right), keyboard handlers (Space: next when allowed, P: pause/resume, S: score round). 15s reading delay shows "Get ready..." overlay before answer buttons enable.
- 5. Build Team UI pages**
 - `/`: Landing with game code input, auto-fill from `?code=` query param from QR scan
 - `/olav/[gamecode]`: Team name editor (disabled after "Ready"), large A/B/C/D buttons with selected state highlight, "Submit Response" → confirmation

PROCEED FROM PLAN

Start Implementation | Open in Editor

Add Context...

Outline the goal or problem to research

Plan Claude Opus 4.5

What was my workflow? (con't)

- Building took a good bit of time and hit context cap a few times.
- This required me to manually advance it during those (~5-8 minutes or so).
- Once done, I requested a deployment guide to test it on local desktop.
- This did take a bit of tweaking to get right (local and cloud).

review DEPLOYMENT_GUIDE.md X

Trivia Game - Complete Deployment Guide

This guide walks you through deploying the Trivia Game application from scratch, including setting up the database, deploying the servers, and configuring your GoDaddy domain.

Table of Contents

1. Prerequisites
2. Local Development Setup
3. Production Database (Supabase)
4. Deploy Socket.IO Server (Railway)
5. Deploy Next.js App (Vercel)
6. Configure GoDaddy Domain
7. Final Testing
8. Maintenance & Updates

Prerequisites

Before starting, ensure you have:

- **Node.js 18+** installed ([download](#))
- **Git** installed ([download](#))
- **GitHub account** (for deployment integrations)
- **GoDaddy account** with a domain you want to use
- **Credit card** (Railway requires payment info, but has a free tier)

Local Development Setup

Step 1: Clone and Install

```
# Clone the repository (or navigate to your existing folder)
cd trivia-game-project

# Install main app dependencies
npm install

# Install socket server dependencies
cd socket-server
npm install
cd ..
```

Step 2: Create Environment Files

Main app - Create `.env.local` in the project root:

What was my workflow? (con't)

See this in action

The screenshot displays a developer's workflow in a multi-pane environment. On the left, the GitHub web interface for the 'trivia-game-project' repository is visible, showing a list of files and a commit history. The central pane shows a code editor with the 'page.jsx' file open, containing JavaScript code for a trivia game. The right pane shows a terminal window with the output of a build command, including file sizes and timestamps. The bottom of the screen shows a Windows taskbar with various application icons and the system clock.

GitHub Repository: trivia-game-project

- Files: README_files/libs, prisma, socket-server, src, env.example, .gitignore, DEPLOYMENT_GUIDE.md, README.html, README.md, Test_question_set.csv, full_question_set.csv, next.config.js, package-lock.json, package.json, postcss.config.js, tailwind.config.ts, trivia_plan.md.txt, tsconfig.json
- Commits: 31

Code Editor: page.jsx

```
export default function StartGamePage() {
  const [showArchived, setShowArchived] = useState(false);

  useEffect(() => {
    loadConfigs();
  }, [showArchived]);

  const loadConfigs = async () => {
    setLoading(true);
    const data = await getTriviaConfigs(showArchived);
    setConfigs(data);
    // Only auto-select if we don't have a selection and there are non-archived
    const activeConfigs = data.filter(c => !c.isArchived);
    if (activeConfigs.length > 0 && !selectedConfigId) {
      setSelectedConfigId(activeConfigs[0].id);
    }
    setLoading(false);
  };

  const handleDelete = async (id: string) => {
    const result = await deleteTriviaConfig(id);
    if (result.success) {
      await loadConfigs();
    } else {
      setError(result.error || 'Failed to delete');
    }
    setDeleteConfirmId(null);
  };

  const handleArchive = async (id: string, archive: boolean) => {
    const result = await archiveTriviaConfig(id, archive);
    if (result.success) {
      if (selectedConfigId === id && archive) {
        setSelectedConfigId('');
      }
      await loadConfigs();
    } else {
      setError(result.error || 'Failed to archive');
    }
    setArchiveConfirmId(null);
  };
}
```

Terminal Output:

```
PS C:\Users\janev\OneDrive\trivia-game-project> dir
-File-
1/12/2026 10:02 PM 1498 package.json
-File-
1/12/2026 9:12 PM 89 postcss.config.js
-File-
1/12/2026 9:44 PM 39488 README.html
-File-
1/12/2026 8:31 AM 2214 README.md
-File-
1/12/2026 10:07 PM 2135 tailwind.config.ts
-File-
1/12/2026 11:37 PM 325 Test_question_set.csv
-File-
1/12/2026 8:41 PM 7827 trivia_plan.md.txt
-File-
1/12/2026 9:12 PM 621 tsconfig.json
```